



**FACULTY OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY
DEPARTMENT OF COMMUNICATION TECHNOLOGY AND NETWORK
SEMESTER II 2025/2026**

COURSE : BACHELOR OF COMPUTER SCIENCE (COMPUTER NETWORK)
WITH HONOURS

COURSE CODE : CSC4202-1 (DESIGN AND ANALYSIS OF ALGORITHMS)

GROUP : GROUP 3

PROJECT TITLE : SCHOOL EMERGENCY EVACUATION ROUTE PLANNING

LECTURER : DR NUR ARZILAWATI MD YUNUS

Name	Matric Number
MUHAMMAD FITRI BINTI MASHARIL	224512
MUHAMMAD FARHAN BIN SAZALI	223293
MUHAMMAD HAZIQ HAKIM BIN YAZID	222562
NUR SHAHIERA BINTI SHAMADI SHAH	224328

Table of Content

1.0 INTRODUCTION.....	3
2.0 PROBLEM STATEMENT AND SCENARIO DEVELOPMENT.....	4
2.1 Geographical Setting.....	5
2.2 Disaster Scenario: Class Fire.....	10
2.3 Damage and Impact.....	11
2.5 Goal and Expected Output.....	15
3.0 ALGORITHM COMPARISON.....	16
3.3 Dynamic Programming (DP).....	17
3.4 Graph Traversal (BFS/DFS).....	17
3.5 Greedy Algorithm: Dijkstra's Algorithm (Selected Solution).....	18
4.0 DIJKSTRA'S ALGORITHM AND TESTING.....	19
4.1 Algorithm Overview.....	20
4.4 Java Implementation.....	24
The implementation of Dijkstra's Algorithm is mainly performed within the <code>dijkstra()</code> function. This function is responsible for calculating the shortest evacuation route from every school location to the designated safe area, Padang (I). The implementation combines the standard Dijkstra shortest-path procedure with an additional fire avoidance mechanism to ensure that evacuation routes do not pass through hazardous locations.	
Initialization Phase.....	24
The algorithm begins by initializing the distance and parent arrays.	
The <code>distance[]</code> array stores the shortest known distance from Padang to every location. Initially, all vertices are assigned infinity to indicate that no route has been discovered. The safe area, Padang, is assigned a distance of zero because it serves as the destination node.	
The <code>parent[]</code> array is used to store predecessor information, allowing the complete evacuation route to be reconstructed after the shortest distances have been determined.	
	24
5.0 CORRECTNESS AND COMPLEXITY ANALYSIS.....	28
5.1 Correctness of Dijkstra's Algorithm.....	28
5.2 Proof of Correctness.....	28
5.3 Theoretical Step-by-Step Working.....	29
5.4 Time Complexity Analysis.....	33
CODE.....	35
RESULT.....	40

1.0 INTRODUCTION

Emergency evacuation planning is a vital part of disaster risk reduction, especially in high-density environments like schools. The goal of an evacuation plan is to ensure that students and staff can react appropriately to early warnings and reach safety as quickly as possible. However, managing a real-life emergency is difficult because successful management requires decision-making and action within a very short time frame of seconds. In a crisis, human emergency commanders often face high levels of stress and limited time to interpret complex data, which can lead to mistakes or "unsubstantiated heuristic reasoning" essentially, making guesses based on instinct rather than facts. This highlights the need for automated support systems that can help coordinate movements efficiently.

A major challenge during an emergency is human behavior. Research shows that people who are panicking tend to choose familiar main roads to escape without realizing those paths might become overcrowded. This behavior often results in total congestion, making the evacuation slower and more dangerous. Additionally, traditional static exit signs may be ineffective or even lead people toward hazards if the situation changes during a disaster. Therefore, a successful evacuation strategy needs to consider not just the shortest route, but also the safest route by accounting for moving hazards like fire or smoke.

In the field of Design and Analysis of Algorithms, these problems are often solved using graph theory. A school building can be modeled as a Geometric Network Graph (GNG), where rooms and hallways are represented as "nodes" (vertexes) and the navigable connections between them are "edges". Dijkstra's algorithm is one of the most famous and effective methods for finding the shortest path between a starting node, such as a classroom, and a finish node, such as a safe assembly point. While the original algorithm usually calculates simple distance, it can be modified to use a "cost function" that includes other factors like travel time, proximity to fire, and physical blockages from structural damage.

This project focuses on the design and analysis of an evacuation system for a school environment using a modified Dijkstra algorithm. By using Building Information Modeling (BIM) data to map the school layout and applying algorithmic pathfinding, the system aims to find the "lowest cost" path for every room. This approach ensures that occupants are guided away from dangerous areas along the most efficient routes, ultimately increasing the chances of a successful and safe evacuation.

2.0 PROBLEM STATEMENT AND SCENARIO DEVELOPMENT

This project focuses on the development of a School Fire Evacuation Planning System that assists students, teachers, and school staff in identifying the shortest and safest evacuation routes during a fire emergency. Fire incidents in schools can spread rapidly and create panic, making it difficult for occupants to determine the most appropriate evacuation path. Delays in evacuation may increase the risk of injuries, congestion, and exposure to hazardous areas.

To address this issue, the school environment is modeled as a weighted graph where buildings and important locations are represented as vertices, while pathways connecting these locations are represented as edges. The objective is to determine the shortest safe route from each location to the designated assembly area, Padang (I), while avoiding areas affected by fire. By applying Dijkstra's Algorithm, the system can generate optimized evacuation routes that support faster and safer emergency responses.

2.1 Geographical Setting

The proposed scenario is situated within a secondary school environment consisting of several interconnected buildings and facilities. The school compound contains academic buildings, administrative offices, student facilities, assembly areas, and entrance gates connected through a network of walkways.

For this project, the school layout consists of the following locations:

Table 2.1: Nodes

Node	Real Location
A	Bilik Gerakan/Stor PMTIA
B	Kantin
C	Pra 1/2/3
D	Tapak Perhimpunan
E	Pejabat (Upper Floor)
F	Dewan (Ground Floor)
G	Surau/Pusat Sumber
H1	Blok Kelas 4, 4A-4C (Single Level)
H2	Blok Utama 1-3,5,6 (3 level)
I	Padang
J	Pintu Masuk Kiri
K	Pintu Masuk Kanan

Table 2.2: Corridor Connections (Edges)

Edge	Distance (m)
A → B	10
B → C	8
B → D	6
A → G	20
D → E	12
E → F	5
F → I	15
G → I	18
C → H1	18
H1 → H2	10
H1 → I	20
H2 → I (Left Stairs)	15
H2 → I (Right Stairs)	18
I → J	25
I → K	20

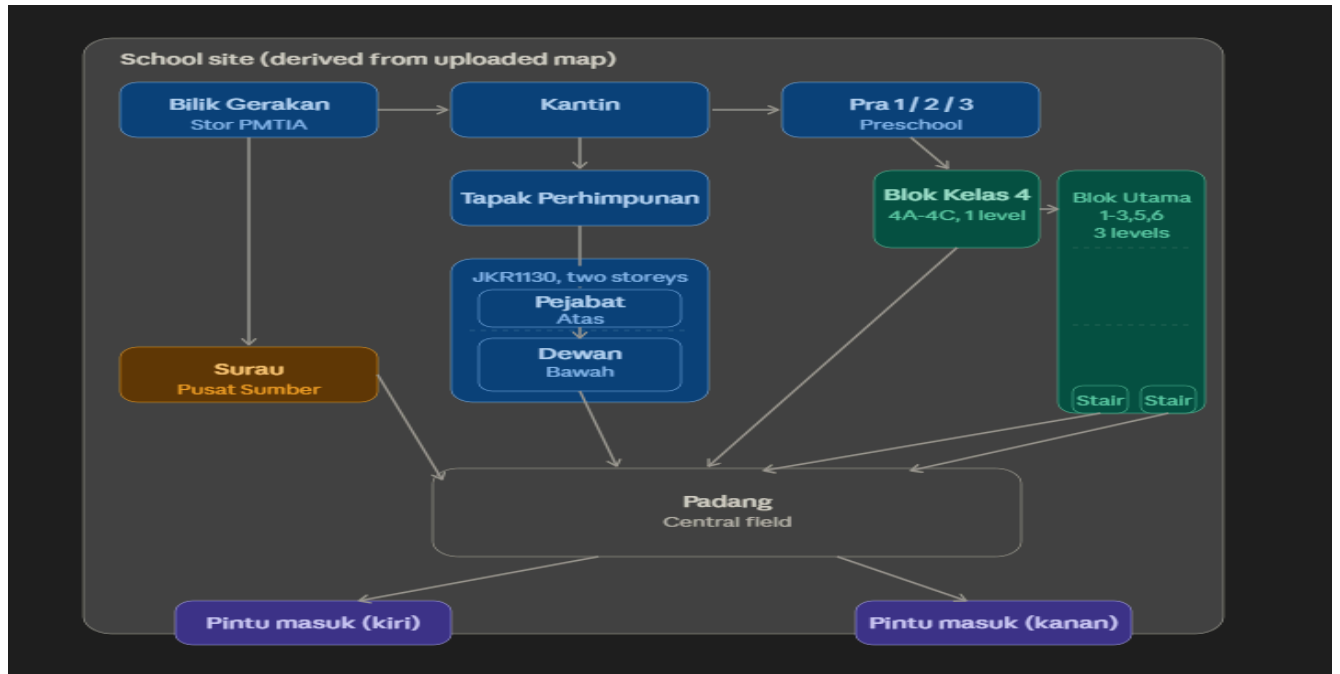
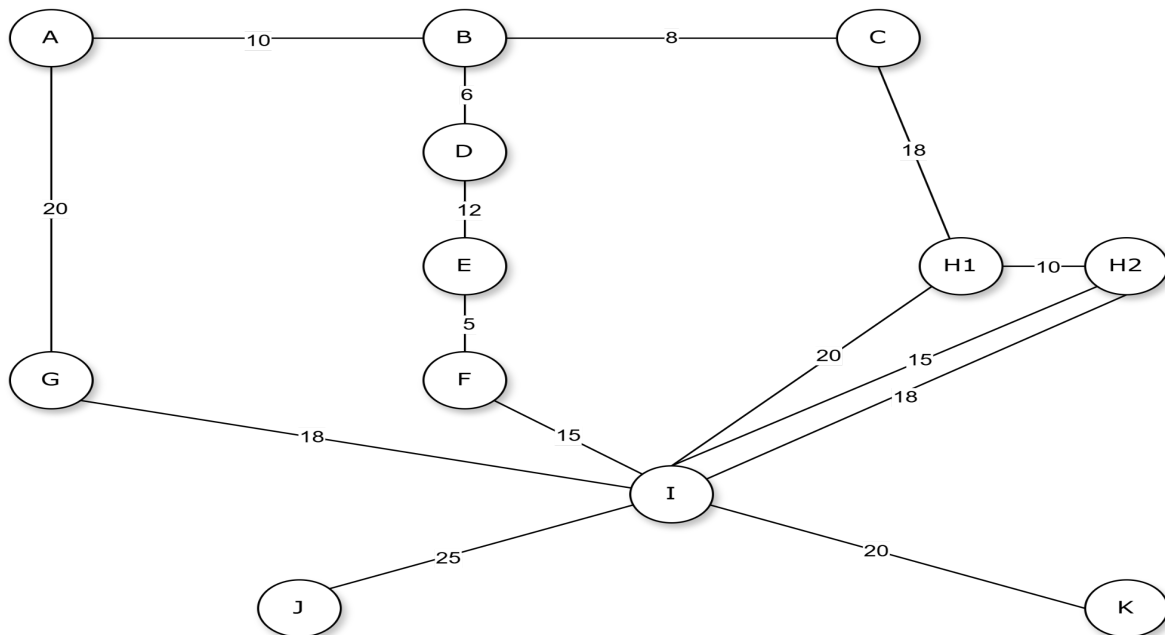


Figure 2.1



The designated evacuation assembly area is Padang (I), which is an open and spacious field capable of accommodating all students and staff during emergencies. The pathways between locations vary in distance and provide multiple possible evacuation routes.

The school environment can therefore be represented as a weighted graph $G(V,E)$, where:

- V represents school locations.
- E represents connecting pathways.
- Edge weights represent walking distances between locations.

This graph representation enables the evacuation problem to be transformed into a shortest-path optimization problem.

2.2 Disaster Scenario: Class Fire

The disaster scenario considered in this project is a fire outbreak that originates from Blok Utama (H1). The fire may be caused by electrical faults, overheating equipment, short circuits, or other classroom-related accidents. Once a fire occurs, smoke and flames can spread quickly through nearby corridors and connected buildings, creating dangerous conditions for occupants.

In this scenario, Blok Utama is assumed to be the initial fire source. As the fire develops, the affected building becomes unsafe and can no longer be used as part of any evacuation route. Students and staff located in surrounding areas must immediately evacuate to the designated assembly area.

The emergency creates several challenges:

- Occupants may not know the safest evacuation route.
- Some routes may pass through dangerous areas.
- Multiple alternative routes may exist.
- Panic situations can lead to poor decision-making.
- Large numbers of students may need to evacuate simultaneously.

Therefore, an automated route-planning mechanism is required to quickly identify the shortest safe path to the assembly area.

2.3 Damage and Impact

A fire emergency within a school environment can cause significant consequences to both people and infrastructure. The severity of the impact depends on the fire location, spread rate, and effectiveness of evacuation procedures.

Safety Impact

The most critical impact involves the safety of students, teachers, and staff. Individuals located near the fire source face increased risks of:

- Burns and physical injuries.
- Smoke inhalation.
- Panic and confusion.
- Delayed evacuation.

Infrastructure Damage

Fire incidents may damage:

- Classrooms.
- Electrical systems.
- Learning equipment.
- School documents and records.
- Building structures.

In severe cases, affected facilities may become unusable and require extensive repairs.

Evacuation Challenges

Without a proper evacuation strategy, several problems may occur:

- Congestion at evacuation routes.
- Bottlenecks at building exits.
- Increased evacuation time.
- Exposure to hazardous areas.
- Difficulty locating safe exits.

These challenges highlight the need for a systematic evacuation planning mechanism that can quickly determine the safest available routes.

2.4 Why This Needs an Optimal Algorithm

During a fire emergency, every second is important. Selecting a route that is longer than necessary can increase evacuation time and expose occupants to additional risks. Therefore, relying solely on human judgement may not always produce the safest or most efficient evacuation decision.

The evacuation problem contains several characteristics that make it suitable for algorithmic optimization:

- Multiple locations require evacuation simultaneously.
- Multiple possible routes exist between locations.
- Some locations become inaccessible due to fire.
- The safest route is not always visually obvious.
- Route decisions must be made quickly.

An optimal algorithm is required because it can evaluate all possible routes and determine the shortest safe path automatically.

Without an Optimization Algorithm

- Occupants may choose inefficient routes
- Evacuation times may increase.
- Congestion may occur.
- Dangerous routes may be selected.

With an Optimization Algorithm

- * Shortest routes are identified automatically.
- * Unsafe locations are avoided.
- * Evacuation time is minimized.
- * Decision-making becomes faster and more reliable.

Dijkstra's algorithm is particularly suitable because it is specifically designed to solve shortest-path problems in weighted graphs. Since all pathway distances are positive values, the algorithm guarantees an optimal solution while maintaining efficient execution.

2.5 Goal and Expected Output

The primary goal of this project is to develop a decision-support system capable of generating the shortest and safest evacuation routes during a school fire emergency.

The system aims to:

- Model the school environment as a weighted graph.
- Exclude fire-affected locations from route computation.
- Determine the shortest safe route to Padang.
- Minimize evacuation distance.
- Provide clear evacuation instructions.

The expected output of the system includes:

Matrix	Description	Example	
Route Information	The complete evacuation path from each location to Padang.	A (Bilik Gerakan) → G (Surau) → I (Padang)	
Distance Information	The total evacuation distance.	Total Distance = 38 meters	
Safety Status	The system indicates whether the route is safe or inaccessible.	Status = SAFE	
Risk Classification	The system classifies routes according to travel distance.	Distance	Risk Level
		0 - 20 m	Low
		21 -40 m	Medium
		Above 40 m	High

The final output provides school administrators and emergency responders with clear evacuation guidance, enabling faster and safer movement towards the designated assembly area during fire emergencies.

3.0 ALGORITHM COMPARISON

Before selecting Dijkstra's Algorithm as the solution for the emergency evacuation route planning, several algorithms in different categories were evaluated based on their suitability for the scenario. The school emergency evacuation problem requires finding the shortest safe path between locations in a weighted network where certain edges (corridors) can be blocked. The analysis reviews the strengths and weaknesses of each algorithm in this context.

3.1 Sorting Algorithms

Sorting algorithms such as Bubble Sort, Merge Sort, and Quick Sort are designed to reorder a list of elements based on a comparison key. While they are efficient for organizing data (with complexities ranging from $O(n^2)$ to $O(n \log n)$), they operate on flat sequences and have no concept of connectivity or graph topology. In the school evacuation problem, knowing which individual corridor is the shortest does not tell us whether it forms a valid connected path from a classroom to the assembly area. Sorting algorithms cannot determine reachability between nodes, handle blocked routes, or produce a sequence of locations to traverse. They are therefore entirely unsuitable for this routing problem.

3.2 Divide and Conquer (DAC)

The Divide and Conquer paradigm solves problems by recursively splitting them into independent, non-overlapping subproblems. Classic examples include Merge Sort and Binary Search. However, the school evacuation graph is a highly interconnected network where paths share nodes and edges. If the school map were arbitrarily divided into two halves, the corridors connecting those halves, which may be critical evacuation routes that would be severed. There is no natural clean division of a graph that preserves path continuity. Additionally, DAC does not account for edge weights or blocked paths. For these reasons, DAC is unsuitable for this scenario.

3.3 Dynamic Programming (DP)

Dynamic Programming solves optimization problems by breaking them into overlapping subproblems and storing intermediate results. The Floyd-Warshall algorithm, a DP-based all-pairs shortest path algorithm, runs in $O(V^3)$ time and computes the shortest path between every pair of nodes. While this is theoretically applicable to the evacuation graph, it is overkill for our scenario. We do not need shortest paths between every pair of school locations, we only need the shortest path from each location to the designated safe area, Padang (I). Computing unnecessary paths wastes time and memory, making DP impractical for real-time emergency response. Furthermore, $O(V^3)$ complexity becomes increasingly slow as the network grows.

3.4 Graph Traversal (BFS/DFS)

Breadth-First Search (BFS) can find the shortest path in an unweighted graph by exploring nodes layer by layer. However, the school evacuation map is a weighted graph, where corridors have different distances, and the shortest path in terms of number of hops is not necessarily the shortest in terms of physical distance. For example, the route $A \rightarrow G \rightarrow J$ (38 m) uses fewer hops than $A \rightarrow B \rightarrow D \rightarrow E \rightarrow F \rightarrow J$ (48 m), and BFS would correctly prefer the former, but only by coincidence of the graph structure. In general, BFS cannot differentiate between a 5-metre and a 20-metre corridor, making it unreliable for distance-based evacuation planning. Depth-First Search (DFS) is even less appropriate as it does not guarantee shortest paths at all.

3.5 Greedy Algorithm: Dijkstra's Algorithm (Selected Solution)

Dijkstra's Algorithm is a greedy shortest-path algorithm that iteratively selects the unvisited node with the smallest known distance from the source and relaxes its neighbours. It operates correctly on any weighted graph with non-negative edge weights, which is guaranteed in our scenario since all corridor distances are physical measurements and cannot be negative.

The key advantages that make Dijkstra's Algorithm the optimal choice for this scenario are as follows. First, it handles weighted edges, allowing the system to differentiate between corridors of different lengths. Second, it can skip blocked nodes — in this implementation, the `blocked[]` array causes the algorithm to skip H1 (Blok Utama), the fire source, when exploring neighbours. Third, it computes the shortest path from all nodes to the safe area in a single run by running from the destination node (Padang) outward, giving complete evacuation guidance for every location. Fourth, its time complexity of $O(V^2)$ using an array-based priority queue is efficient for a 12-node graph and can recalculate routes near-instantly if corridor conditions change. Table 3.1 below summarises the comparison.

Paradigm	Handle Weights	Handles Blocked Nodes	Suitable For The Scenario
Sorting	No	No	Unsuitable
Divide & Conquer	No	No	Unsuitable
Dynamic Programming (Floyd-Warshall)	Yes	Partially	Too Slow
Graph Traversal (BSF)	No	Partially	Inadequate
Greedy (Dijkstra's)	Yes	Yes	Excellent

Table 3.1

4.0 DIJKSTRA'S ALGORITHM AND TESTING

This section presents the implementation and testing of Dijkstra's Algorithm for the proposed School Fire Evacuation Planning System. The objective of the algorithm is to determine the shortest and safest evacuation route from every location within the school compound to the designated assembly area, Padang (I), during a fire emergency. The implementation of Dijkstra's Algorithm in the School Fire Evacuation Planning System focuses on the following key aspects:

- Shortest Path Optimization – Determines the minimum evacuation distance from every school location to Padang (I).
- Fire Hazard Avoidance – Excludes H1 (Blok Utama) from route computation because it represents the fire source.
- Graph-Based Modeling – Represents the school environment as a weighted graph consisting of vertices and edges.
- Priority Queue Utilization – Improves route computation efficiency by always selecting the vertex with the smallest tentative distance.
- Route Reconstruction – Uses the parent array to generate complete evacuation routes instead of displaying distance values only.
- Safety-Oriented Decision Making – Ensures that generated routes are both optimal and safe for evacuation purposes.
- Testing and Validation – Verifies the correctness of the generated routes and confirms that the shortest safe paths are successfully identified.

The implementation demonstrates how Dijkstra's Algorithm can be applied in a real-world emergency evacuation scenario to support rapid decision-making and improve evacuation efficiency during fire incidents.

4.1 Algorithm Overview

The algorithm belongs to the Greedy Algorithm paradigm. At each iteration, Dijkstra's Algorithm selects the unvisited vertex with the smallest known distance and expands its neighbouring vertices. This process continues until all reachable vertices have been processed. Since all edge weights are non-negative, the algorithm guarantees that the shortest path found is globally optimal. The algorithm is widely used in route planning, transportation systems, network routing protocols, emergency response systems, and navigation applications because of its ability to efficiently determine the shortest path between locations connected within a weighted network.

The school map is represented as a weighted graph $G(V,E)$, where:

- V represents the set of vertices (school locations).
- E represents the set of edges (walkways).
- Edge weights represent walking distances.
- Padang (I) acts as the evacuation destination.
- H1 (Blok Utama) acts as the fire source and is excluded from route computation.

School Component	Graph Representation
Bilik Gerakan	Vertex
Kantin	Vertex
Pra 1/2/3	Vertex
Tapak Perhimpunan	Vertex
Pejabat Atas	Vertex
Dewan Bawah	Vertex
Surau	Vertex
Blok Utama (Fire source)	Blocked Vertex
Blok Kelas 4	Vertex
Padang	Destination Vertex
Walkway	Edge
Walking Distance	Edge Weight

4.2 Pseudocode

The following pseudocode describes the operation of Dijkstra's Algorithm within the School Fire Evacuation Planning System.

Input:

- School graph $G(V,E)$
- Safe Area = Padang (I)
- Fire Source = H1 (Blocked)

Output:

- Shortest evacuation route
- Total evacuation distance
- Route status

1. For each vertex v in graph G
 $\text{distance}[v] \leftarrow \infty$
 $\text{parent}[v] \leftarrow \text{NIL}$
 $\text{visited}[v] \leftarrow \text{FALSE}$
2. $\text{distance}[\text{Padang}] \leftarrow 0$
3. Mark H1 as blocked
4. Insert Padang into Priority Queue
5. While Priority Queue is not empty

$u \leftarrow$ vertex with minimum distance

$\text{visited}[u] \leftarrow \text{TRUE}$

For each neighbour v of u

 If v is blocked

 Continue

$\text{NewDistance} \leftarrow$

$\text{distance}[u] + \text{weight}(u,v)$

 If $\text{NewDistance} < \text{distance}[v]$

$\text{distance}[v] \leftarrow \text{NewDistance}$

```
parent[v] ← u
```

```
Insert v into Priority Queue
```

```
6. End While
```

```
7. Reconstruct routes using parent[]
```

```
8. Display route, distance, and status
```

```
9. End
```

The pseudocode illustrates the iterative nature of Dijkstra's Algorithm. During each iteration, the algorithm selects the vertex with the smallest tentative distance and performs an edge relaxation process. Edge relaxation is the mechanism used to determine whether a newly discovered route is shorter than the currently stored route.

Whenever a shorter route is found, the distance value and predecessor information are updated. This process continues until no shorter paths can be discovered. The parent array is then used to reconstruct the final evacuation route from each location to Padang.

4.3 Flowchart

The flowchart of the proposed system illustrates the overall execution process of Dijkstra's Algorithm within the school evacuation scenario. The process begins by constructing the weighted graph representation of the school map. All school locations and connecting pathways are inserted into the graph along with their corresponding distance values.

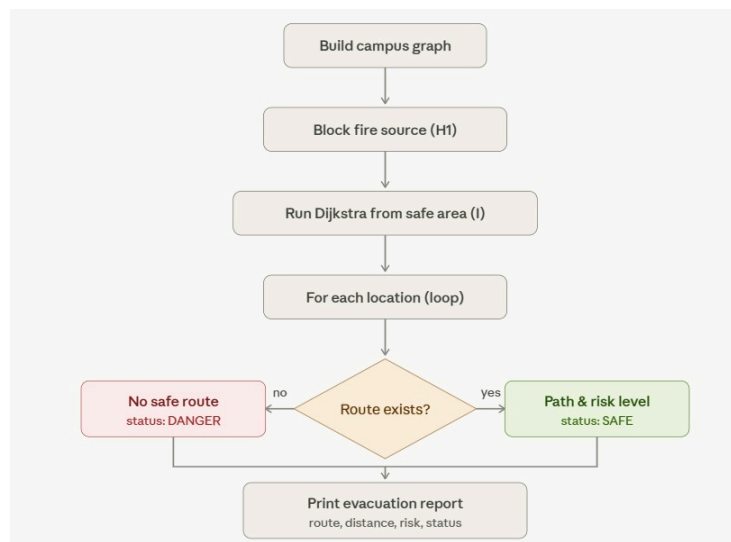
Next, H1 (Blok Utama) is designated as the fire source and is marked as a blocked node. This prevents the algorithm from generating routes that pass through hazardous areas.

The system then initializes the distance array by assigning infinity to all locations except Padang, which is assigned a distance of zero. Padang is inserted into a Priority Queue as the starting point of route computation.

The algorithm repeatedly performs the following steps:

1. Select the vertex with the smallest tentative distance.
2. Examine all neighbouring vertices.
3. Ignore blocked locations.
4. Calculate new tentative distances.
5. Update distance and parent information whenever a shorter route is discovered.
6. Insert updated vertices into the Priority Queue.

The operational flow of the system can be summarized as follows:



4.4 Java Implementation

The implementation of Dijkstra's Algorithm is mainly performed within the `dijkstra()` function. This function is responsible for calculating the shortest evacuation route from every school location to the designated safe area, Padang (I). The implementation combines the standard Dijkstra shortest-path procedure with an additional fire avoidance mechanism to ensure that evacuation routes do not pass through hazardous locations.

Initialization Phase

The algorithm begins by initializing the distance and parent arrays.

```
Arrays.fill(distance, Integer.MAX_VALUE);  
Arrays.fill(parent, -1);  
  
distance[safeArea] = 0;
```

The `distance[]` array stores the shortest known distance from Padang to every location. Initially, all vertices are assigned infinity to indicate that no route has been discovered. The safe area, Padang, is assigned a distance of zero because it serves as the destination node.

The `parent[]` array is used to store predecessor information, allowing the complete evacuation route to be reconstructed after the shortest distances have been determined.

Priority Queue Implementation

```
PriorityQueue<Node> pq = new PriorityQueue<>();  
  
pq.add(new Node(safeArea, 0));
```

A Priority Queue is used to implement the greedy strategy of Dijkstra's Algorithm. The queue automatically sorts vertices according to their distance values and ensures that the vertex with the smallest tentative distance is always processed first.

This improves efficiency because the algorithm does not need to repeatedly search through all vertices to find the smallest distance.

Main Dijkstra Iteration

The core implementation of Dijkstra's Algorithm is located inside the following loop:

```
while (!pq.isEmpty()) {  
    Node current = pq.poll();  
    int u = current.vertex;  
    ...  
}
```

The poll() operation removes and returns the vertex with the smallest distance from the Priority Queue. This is the greedy selection step of Dijkstra's Algorithm. For example, if the queue contains:

Vertex	Distance
F	15
G	18
A	38

The algorithm automatically selects vertex F because it has the smallest distance value.

Neighbour Exploration

After selecting the current vertex, the algorithm explores all neighbouring locations.

```
for (Edge edge : graph.get(u))
```

This loop visits every vertex directly connected to the current location. For example, if the current location is F (Dewan Bawah), the algorithm examines all pathways connected to F and evaluates whether they provide a shorter route to Padang.

Edge Relaxation Process

The most important operation in Dijkstra's algorithm is edge relaxation.

```
int newDistance =  
distance[u] + edge.weight;
```

This statement calculates the distance of an alternative route through the current vertex. For example:

Current Distance	Edge Weight
15	5

The algorithm then compares this value against the currently stored distance.

```
if (newDistance < distance[v])
```

If the newly calculated distance is smaller, the route is improved.

```
distance[v] = newDistance;  
parent[v] = u;
```

This process ensures that only the shortest route discovered so far is retained.

Route Reconstruction

After all vertices have been processed, the shortest evacuation route is reconstructed using the parent array.

```
buildPath(i, parent);
```

The function traces the predecessor relationships stored during the relaxation process and generates the complete evacuation path. This allows the system to display not only the shortest distance but also the complete evacuation path that students and staff should follow.

Key Functions Related to Dijkstra's Algorithm

Function / Code Segment	Purpose
dijkstra()	Main shortest-path computation
PriorityQueue<Node>	Selects the vertex with minimum distance
pq.poll()	Retrieves the nearest vertex
for (Edge edge : graph.get(u))	Explores neighbouring vertices
if (blocked[v]) continue;	Skips fire-affected locations
newDistance = distance[u] + edge.weight	Calculates alternative routes
if (newDistance < distance[v])	Compares route distances
distance[v] = newDistance	Updates shortest distance
parent[v] = u	Stores route information
buildPath()	Reconstructs evacuation routes

5.0 CORRECTNESS AND COMPLEXITY ANALYSIS

5.1 Correctness of Dijkstra's Algorithm

The correctness of Dijkstra's Algorithm is grounded in the greedy-choice property which means once a node is finalized (marked as visited), its recorded shortest distance from the source is guaranteed to be globally optimal and will never be improved by any other future path.

In this implementation, the algorithm is run from the safe area, Padang (I), as the source node. The *distance[]* array is initialized with *Integer.MAX_VALUE* (representing infinity) for all nodes, except the source which is set to 0. The *parent[]* array records the predecessor of each node to allow full path reconstruction. A *PriorityQueue<Node>* ordered by distance ensures the node with the smallest tentative distance is always processed next.

5.2 Proof of Correctness

Let node u be selected as the unvisited node with the minimum tentative distance $dist[u]$. Suppose a shorter path to u exists through some other unvisited node v .

For this to be true: $dist[v] + weight(v, u) < dist[u]$

However, since u was selected as the minimum-distance unvisited node: $dist[v] \geq dist[u]$

And since all edge weights are non-negative: $dist[v] + weight(v, u) \geq dist[u]$

This directly contradicts the assumption. Therefore $dist[u]$ is optimal the moment u is finalized. By induction, this holds for every node processed.

5.3 Theoretical Step-by-Step Working

The algorithm is executed from source = I (Padang), with H1 (Blok Utama) blocked due to fire. All edges touching H1, namely $C \leftrightarrow H1$, $H1 \leftrightarrow H2$, and $H1 \leftrightarrow I$, are removed from consideration.

Initialization:

Node	A	B	C	D	E	F	G	H1		H2	I	J	K
dist	∞	∞	∞	∞	∞	∞	∞	BLOCKED		∞	0	∞	∞
prev	-	-	-	-	-	-	-	-		-	-	-	-

Figure 5.0

Iteration 2 — Process F (dist = 15, tie-broken with H2):
Relax neighbours of F: E

Node	Calculation	New dist	prev
E	$15 + 5 = 20$	20	F

Figure 5.1

Iteration 4 — Process G (dist = 18):
Relax neighbours of G: A

Node	Calculation	New dist	prev
A	$18 + 20 = 38$	38	G

Figure 5.2

Iteration 5 — Process E (dist = 20):

Relax neighbours of E: D

Node	Calculation	New dist	prev
D	$20 + 12 = 32$	32	E

Figure 5.3

Iteration 6 — Process K (dist = 20):

No unvisited neighbours that improve. No updates.

Figure 5.4

Iteration 7 — Process D (dist = 32):

Relax neighbours of D: B

Node	Calculation	New dist	prev
B	$32 + 6 = 38$	38	D

Figure 5.5

Iteration 8 — Process J (dist = 25):

No unvisited neighbours that improve. No updates.

Figure 5.6

Iteration 9 — Process A (dist = 38, tie-broken with B):

Neighbours of A: $B \rightarrow 38 + 10 = 48 > \text{current dist } 38$. No improvement.

Figure 5.7

Iteration 10 — Process B (dist = 38):

Relax neighbours of B: C

Node	Calculation	New dist	prev
C	$38 + 8 = 46$	46	B

Note: B also connects to A ($38 + 10 = 48 > 38$, no update) and D ($38 + 6 = 44 > 32$, no update).

Figure 5.8

Iteration 11 — Process C (dist = 46):

Neighbours of C: H1 → BLOCKED, skip. No updates.

Figure 5.9

Final Theoretical Results:

Location	Shortest dist (m)	Shortest Route (Theoretical)
F (Dewan)	15	F → I
H2 (Blok Kelas 4)	15	H2 → I
G (Surau)	18	G → I
E (Pejabat)	20	E → F → I
K (Pintu Kanan)	20	K → I
J (Pintu Kiri)	25	J → I
D (Tapak Perhimpunan)	32	D → E → F → I
A (Bilik Gerakan)	38	A → G → I
B (Kantin)	38	B → D → E → F → I
C (Pra 1/2/3)	46	C → B → D → E → F → I
H1 (Blok Utama)	BLOCKED	NO ROUTE

Figure 5.10

Comparison: Theoretical vs Code Output					
Location	Theoretical dist (m)	Theoretical Route	Code Output dist (m)	Code Output Route	Match?
F (Dewan)	15	F → I	15	I → F	✓
H2 (Blok Kelas 4)	15	H2 → I	15	I → H2	✓
G (Surau)	18	G → I	18	I → G	✓
E (Pejabat)	20	E → F → I	20	I → F → E	✓
K (Pintu Kanan)	20	K → I	20	I → K	✓
J (Pintu Kiri)	25	J → I	25	I → J	✓
D (Tapak Perhimpunan)	32	D → E → F → I	32	I → F → E → D	✓
A (Billik Gerakan)	38	A → G → I	38	I → G → A	✓
B (Kantin)	38	B → D → E → F → I	38	I → F → E → D → B	✓
C (Pra 1/2/3)	46	C → B → D → E → F → I	46	I → F → E → D → B → C	✓
H1 (Blok Utama)	BLOCKED / NO ROUTE	—	BLOCKED (skipped)	—	✓

Figure 5.11

All 11 results match perfectly between the theoretical hand-traced working and the Java program output. The routes appear reversed in the code output because the algorithm runs from Padang (I) outward which is the *buildPath()* function reconstructs the path by tracing the *parent[]* array backward from each location to the source, which is the standard correct behaviour of Dijkstra's path reconstruction. The underlying distances and node sequences are identical in both directions, confirming that the implementation is correct.

5.4 Time Complexity Analysis

The implementation uses a *PriorityQueue* (min-heap) combined with an adjacency list representation. Let V = number of vertices (12 in this scenario) and E = number of edges (15 in this scenario).

Best Case $O((V + E) \log V)$

The best case occurs when the destination node is adjacent to the source and is resolved immediately. However, since the algorithm must still initialize all V distance values and process entries from the priority queue, the minimum cost of the algorithm is determined by the heap operations. Each node is inserted and extracted from the priority queue at most once in $O(\log V)$ time, and each edge is relaxed at most once in $O(\log V)$ time for the potential queue update. Therefore the best case time complexity is $O((V + E) \log V)$.

For this scenario: $O((12 + 15) \log 12) \approx O(27 \times 3.58) \approx O(97)$, which is computationally negligible.

Average Case $O((V + E) \log V)$

In the average case, the algorithm processes all reachable nodes and relaxes most edges. With the priority queue implementation, each of the E edges triggers at most one `offer()` operation costing $O(\log V)$, and each of the V nodes is extracted once at $O(\log V)$. The average case complexity is therefore also $O((V + E) \log V)$.

Worst Case $O((V + E) \log V)$

The worst case occurs when every node is reachable and every edge must be processed — no nodes are blocked and no early termination occurs. The priority queue may hold up to $O(E)$ entries due to lazy deletion (old entries remain until popped). Each extraction is $O(\log E) = O(\log V^2) = O(2 \log V) = O(\log V)$. All V extractions and E relaxations give a total of $O((V + E) \log V)$.

If the implementation used a simple array-based minimum search instead of a priority queue (as described in the problem statement's complexity section), the time complexity would be $O(V^2)$ — scanning all V nodes to find the minimum at each of the V iterations. For $V = 12$, this gives 144 operations, which is still extremely fast.

Case	Time Complexity
Best Case	$O((V + E) \log V)$
Average Case	$O((V + E) \log V)$
Worst Case	$O((V + E) \log V)$

CODE

```
import java.util.*;

class Edge {
    int dest;
    int weight;

    public Edge(int dest, int weight) {
        this.dest = dest;
        this.weight = weight;
    }
}

class Node implements Comparable<Node> {
    int vertex;
    int distance;

    public Node(int vertex, int distance) {
        this.vertex = vertex;
        this.distance = distance;
    }

    @Override
    public int compareTo(Node other) {
        return Integer.compare(this.distance, other.distance);
    }
}

public class Main {

    static final int V = 12;

    static String[] locationNames = {
        "A (Bilik Gerakan)",
        "B (Kantin)",
        "C (Pra 1/2/3)",
        "D (Tapak Perhimpunan)",
        "E (JKR1130 - Pejabat Atas)",
        "F (Dewan Bawah)",
        "G (Surau / Pusat Sumber)",
        "H1 (Blok Utama)",
        "H2 (Blok Kelas 4)",
        "I (Padang)",
        "J (Pintu Masuk Kiri)",
        "K (Pintu Masuk Kanan)"
    };

    static boolean[] blocked = new boolean[V];
```

```

public static void addEdge(List<List<Edge>> graph,
                           int src,
                           int dest,
                           int weight) {

    graph.get(src).add(new Edge(dest, weight));
    graph.get(dest).add(new Edge(src, weight));
}

public static String getRiskLevel(int distance) {

    if (distance <= 20)
        return "LOW";

    if (distance <= 40)
        return "MEDIUM";

    return "HIGH";
}

public static String buildPath(int vertex, int[] parent) {

    List<String> path = new ArrayList<>();

    while (vertex != -1) {
        path.add(locationNames[vertex]);
        vertex = parent[vertex];
    }

    return String.join("\n | \n", path);
}

public static void dijkstra(List<List<Edge>> graph, int safeArea) {

    int[] distance = new int[V];
    int[] parent = new int[V];

    Arrays.fill(distance, Integer.MAX_VALUE);
    Arrays.fill(parent, -1);

    PriorityQueue<Node> pq = new PriorityQueue<>();

    distance[safeArea] = 0;
    pq.add(new Node(safeArea, 0));

    while (!pq.isEmpty()) {

        Node current = pq.poll();
    }
}

```

```

        int u = current.vertex;

        for (Edge edge : graph.get(u)) {

            int v = edge.dest;

            if (blocked[v])
                continue;

            int newDistance = distance[u] + edge.weight;

            if (newDistance < distance[v]) {

                distance[v] = newDistance;
                parent[v] = u;

                pq.add(new Node(v, newDistance));
            }
        }
    }
}

```

```

System.out.println("=====");
System.out.println("    SCHOOL FIRE EVACUATION PLANNING SYSTEM");
System.out.println("    Using Dijkstra's Shortest Path Algorithm");

```

```

System.out.println("=====\\n");

System.out.println("Fire Source : H1 (Blok Utama)");
System.out.println("Safe Area   : I (Padang)");
System.out.println("Status     : H1 has been blocked.\\n");

```

```

System.out.println("=====");
System.out.println("    EVACUATION ROUTE ANALYSIS");

```

```

System.out.println("=====\\n");

```

```

        for (int i = 0; i < V; i++) {

            if (i == safeArea || blocked[i])
                continue;

            System.out.println("LOCATION : " + locationNames[i]);

            if (distance[i] == Integer.MAX_VALUE) {

                System.out.println("Route    : NO SAFE ROUTE AVAILABLE");
            }
        }
    }
}

```

```

        System.out.println("Status    : DANGER");

    } else {

        System.out.println("\nShortest Route:");
        System.out.println(buildPath(i, parent));

        System.out.println("\nTotal Distance : "
            + distance[i] + " meters");

        System.out.println("Risk Level    : "
            + getRiskLevel(distance[i]));

        System.out.println("Status      : SAFE");
    }

    System.out.println("\n-----\n");
}

System.out.println("=====");
System.out.println("          EVACUATION ANALYSIS COMPLETE");

System.out.println("=====");
}

public static void main(String[] args) {

    List<List<Edge>> graph = new ArrayList<>();

    for (int i = 0; i < V; i++) {
        graph.add(new ArrayList<>());
    }

    // A = Bilik Gerakan
    // B = Kantin
    // C = Pra 1/2/3
    // D = Tapak Perhimpunan
    // E = Pejabat Atas
    // F = Dewan Bawah
    // G = Surau
    // H1 = Blok Utama
    // H2 = Blok Kelas 4
    // I = Padang
    // J = Pintu Masuk Kiri
    // K = Pintu Masuk Kanan

    addEdge(graph, 0, 1, 10); // A-B
    addEdge(graph, 0, 6, 20); // A-G

```

```
addEdge(graph, 1, 2, 8); // B-C
addEdge(graph, 1, 3, 6); // B-D

addEdge(graph, 3, 4, 12); // D-E
addEdge(graph, 4, 5, 5); // E-F

addEdge(graph, 5, 9, 15); // F-I
addEdge(graph, 6, 9, 18); // G-I

addEdge(graph, 2, 7, 18); // C-H1
addEdge(graph, 7, 8, 10); // H1-H2

addEdge(graph, 7, 9, 20); // H1-I
addEdge(graph, 8, 9, 15); // H2-I

addEdge(graph, 9, 10, 25); // I-J
addEdge(graph, 9, 11, 20); // I-K

// FIRE LOCATION
int FIRE_NODE = 7; // H1

// SAFE AREA
int SAFE_AREA = 9; // I

blocked[FIRE_NODE] = true;

dijkstra(graph, SAFE_AREA);
}
```

RESULT

```
=====
SCHOOL FIRE EVACUATION PLANNING SYSTEM
Using Dijkstra's Shortest Path Algorithm
=====
```

Fire Source : H1 (Blok Utama)
Safe Area : I (Padang)
Status : H1 has been blocked.

```
=====
EVACUATION ROUTE ANALYSIS
=====
```

LOCATION : A (Bilik Gerakan)

Shortest Route:

A (Bilik Gerakan)

|

G (Surau / Pusat Sumber)

|

I (Padang)

Total Distance : 38 meters
Risk Level : MEDIUM
Status : SAFE

LOCATION : B (Kantin)

Shortest Route:

B (Kantin)

|

D (Tapak Perhimpunan)

|

E (JKR1130 - Pejabat Atas)

|

F (Dewan Bawah)

|

I (Padang)

Total Distance : 38 meters
Risk Level : MEDIUM
Status : SAFE

LOCATION : C (Pra 1/2/3)

Shortest Route:

C (Pra 1/2/3)

|

B (Kantin)

|

D (Tapak Perhimpunan)

|

E (JKR1130 - Pejabat Atas)

|

F (Dewan Bawah)

|

I (Padang)

Total Distance : 46 meters

Risk Level : HIGH

Status : SAFE

LOCATION : D (Tapak Perhimpunan)

Shortest Route:

D (Tapak Perhimpunan)

|

E (JKR1130 - Pejabat Atas)

|

F (Dewan Bawah)

|

I (Padang)

Total Distance : 32 meters

Risk Level : MEDIUM

Status : SAFE

LOCATION : E (JKR1130 - Pejabat Atas)

Shortest Route:

E (JKR1130 - Pejabat Atas)

|

F (Dewan Bawah)

|

I (Padang)

Total Distance : 20 meters

Risk Level : LOW

Status : SAFE

LOCATION : F (Dewan Bawah)

Shortest Route:
F (Dewan Bawah)

|
I (Padang)

Total Distance : 15 meters

Risk Level : LOW

Status : SAFE

LOCATION : G (Surau / Pusat Sumber)

Shortest Route:
G (Surau / Pusat Sumber)

|
I (Padang)

Total Distance : 18 meters

Risk Level : LOW

Status : SAFE

LOCATION : H2 (Blok Kelas 4)

Shortest Route:
H2 (Blok Kelas 4)

|
I (Padang)

Total Distance : 15 meters

Risk Level : LOW

Status : SAFE

LOCATION : J (Pintu Masuk Kiri)

Shortest Route:
J (Pintu Masuk Kiri)

|
I (Padang)

Total Distance : 25 meters

Risk Level : MEDIUM

Status : SAFE

LOCATION : K (Pintu Masuk Kanan)

Shortest Route:

K (Pintu Masuk Kanan)

|
I (Padang)

Total Distance : 20 meters

Risk Level : LOW

Status : SAFE

=====

EVACUATION ANALYSIS COMPLETE

=====